

Contexte

Vous devez développer une application pour gérer les employés d'une entreprise. Chaque employé est caractérisé par son identifiant, son nom, son département et son salaire.

Objectifs

- Manipuler les différentes collections Java (HashSet, TreeSet, LinkedHashSet, ArrayList, HashMap)
- Utiliser les itérateurs et les énumérateurs
- Comparer les performances des différentes structures de données
- Convertir entre collections et tableaux

1. Classe Employé

Créez une classe `Employe` qui représente les employés avec les attributs suivants :

- `id (int)` : identifiant unique
- `nom (String)` : nom de l'employé
- `departement (String)` : département
- `salaire (double)` : salaire mensuel

Ajoutez :

- Un constructeur avec tous les paramètres
- Les getters et setters
- La méthode `toString()` pour un affichage formaté
- La méthode `equals()` basée sur l'id

2. Les collections Set

a) HashSet

Dans une classe principale `GestionEmployes`, créez une `HashSet<Employe>` nommée `ensembleEmployes` contenant 5 employés avec des données différentes.

b) TreeSet

Copiez le contenu de `ensembleEmployes` dans un `TreeSet<Employe>` nommé `ensembleTries`.
Astuce : Pour que `TreeSet` fonctionne, la classe `Employe` doit implémenter `Comparable<Employe>` et trier par id.

c) Affichage et mesure des performances

Affichez les deux ensembles (`HashSet` et `TreeSet`) en mesurant le temps d'exécution en nanosecondes pour chaque affichage. Utilisez `System.nanoTime()` avant et après l'affichage.

d) LinkedHashSet

Créez une troisième collection `ensembleLinked` de type `LinkedHashSet<Employe>` qui contient tous les employés des deux premières collections (sans doublons).

e) Conversion en tableau

Convertissez `ensembleLinked` en un tableau d'objets `Employe[]` nommé `tableauEmployes`.

3. Les collections Map

a) HashTable

À partir du tableau `tableauEmployes`, extrayez les données selon les règles suivantes :

- Si le salaire de l'employé est > 5000 DH, insérez son nom dans une `ArrayList<String>` nommée `listeNomsEmployes`
- Si le salaire de l'employé est ≤ 5000 DH, insérez dans une `Hashtable<Integer, String>` nommée `mapEmployes` l'id comme clé et le nom comme valeur

b) Recherche dans la Hashtable

- Recherchez si un id donné (par exemple 103) existe dans `mapEmployes`
- Recherchez si un nom donné (par exemple "Mohamed") existe dans `mapEmployes`

c) HashMap

Créez une nouvelle collection `copieMapEmployes` de type `HashMap<Integer, String>` qui contient une copie de `mapEmployes`.

4. Utilisation des itérateurs

1. Utilisez un **itérateur** (Iterator) pour afficher tous les éléments de `listeNomsEmployes`
2. Utilisez un **énumérateur** (Enumeration) pour afficher toutes les valeurs de `mapEmployes`
3. Utilisez la méthode `entrySet()` pour afficher toutes les clés et valeurs de `copieMapEmployes`.

5. Test de performance

Testez et comparez le temps d'affichage entre `mapEmployes` (Hashtable) et `copieMapEmployes` (HashMap) en affichant tous leurs éléments. Calculez la différence de temps en nanosecondes.